Construire CollabDoc sur AWS, du compte vide à la soutenance

Un week-end | Plan gratuit AWS | Une seule région : eu-north-1 | AWS SAM · Lambda · DynamoDB

Ce document raconte, dans l'ordre où il a été réalisé, la construction du projet CollabDoc : un éditeur de textes à plusieurs, déployé sur AWS, dont les liens de téléchargement PDF ne délivrent jamais un exemplaire de trop. Il est écrit pour qu'un développeur **qui n'a jamais utilisé AWS** puisse refaire le même projet en suivant les étapes une à une.

Tout ce qui touche à AWS est détaillé : les comptes, les outils à installer, chaque commande, chaque fichier de configuration et chaque bloc du modèle d'infrastructure. Le code de l'interface (frontend) et des fonctions (backend) n'est **pas** détaillé. C'est votre part du travail, et le rendu visuel vous appartient. Le guide dit en revanche, pour chaque partie du code, **ce qu'elle doit faire** pour que les contraintes du sujet soient tenues. Chaque choix est justifié par une consigne du sujet.

CONVENTIONS DE LECTURE

- Les blocs **Terminal** se tapent dans le terminal de votre ordinateur, à la racine du projet sauf mention contraire. Les lignes grises sont des exemples de sortie.
- Les encadrés **CONSIGNE** rattachent l'étape à une phrase du sujet. Les encadrés **PIÈGE** décrivent un problème réellement rencontré pendant le projet, et sa solution.
- La région utilisée est `eu-north-1` (Stockholm). Si votre compte est dans une autre région, remplacez-la partout et faites `export AWS_REGION=<votre-région>` avant de lancer les scripts.

Les consignes du sujet, et comment ce guide y renvoie

Repère	Ce que dit le sujet
C1	Un lien permet exactement le nombre de téléchargements autorisé, jamais un de plus, quel que soit le nombre de clics simultanés.
C2	Aucune modification acceptée n'est perdue, et tous les participants finissent par voir le même document, même en écrivant au même endroit.
C3	Un lien créé à T et utilisé à T1 donne le document tel qu'il est à T1, sans nouveau lien.
C4	Un rédacteur n'écrit jamais plus que son crédit du jour (10 000 caractères), même depuis plusieurs appareils.
C5	15 000 visiteurs (80 % dans la première minute) pendant que 300 rédacteurs écrivent : chaque visiteur obtient une réponse (télécharger, patienter ou épuisé).
C6	Après le dernier exemplaire, les visiteurs l'apprennent vite, et l'application cesse de délivrer le service.
R-Compte	Votre propre compte AWS, en plan gratuit, toutes les ressources dans une seule région.
R-Clés	Aucune clé d'accès AWS dans le code ou le dépôt.
R-Droits	Chaque composant n'a que les droits dont il a besoin.
R-S3	Les fichiers à télécharger sont conservés dans S3.
R-Pratiques	Appliquer les bonnes pratiques vues en cours.
Livrables	Dépôt Git (code, infrastructure, tests, README), dossier de conception, rapport de test, description de l'API, preuve des coûts.

Le sujet insiste : *« une application simple, mais dont chaque choix est compris et justifié »*. C'est pourquoi chaque étape ci-dessous dit pourquoi elle existe.

1. Vendredi soir : traduire le sujet en mécanismes AWS

Avant d'ouvrir la console AWS, on prend une feuille et on associe chaque contrainte à un mécanisme qui la **garantit**, et pas seulement qui la rend probable. Ce tableau a guidé tout le projet ; il deviendra le cœur du dossier de conception.

Contrainte	Mécanisme choisi	Pourquoi celui-là
C1 un exemplaire de trop	Compteur `remaining` dans DynamoDB, décrémenté par une **écriture conditionnelle** : `SET remaining = remaining - 1` si `remaining > 0`.	DynamoDB applique les écritures sur un même élément l'une après l'autre et évalue la condition sur la valeur à jour : avec N exemplaires, exactement N écritures réussissent. Pas de « lire puis écrire », qui laisse une fenêtre de course.
C2 aucune perte, convergence	Un **CRDT** (bibliothèque Yjs) : chaque modification est stockée dans DynamoDB **avant** d'être confirmée, puis diffusée par WebSocket.	Les modifications d'un CRDT se fusionnent dans n'importe quel ordre, même en double, et donnent le même texte. Le serveur n'a qu'à ne rien perdre, ce qui est compatible avec des fonctions Lambda sans mémoire.
C3 le document à T1	Le PDF est produit **au moment du téléchargement**, mis en cache dans S3 sous le numéro de révision du document.	Le lien ne contient pas de fichier, seulement un compteur : il sert toujours l'état courant.
C4 crédit multi-appareils	Une **transaction DynamoDB** : « stocker la modification » + « ajouter n à `used` si `used ≤ 10 000 − n` ».	Un seul compteur par personne et par jour, quel que soit l'appareil ; tout ou rien.
C5 le pic	CloudFront pour les pages, limitation de débit d'API Gateway, **concurrence Lambda réservée**, DynamoDB à la demande.	Absorber le pic sans serveur à dimensionner, et cloisonner les visiteurs pour que les rédacteurs continuent d'écrire.
C6 fin de distribution	Réponse `410` immédiate, statut du lien mis en cache 2 s par CloudFront, flux DynamoDB → fonction → e-mail SNS.	Plus aucun PDF généré après épuisement ; l'e-mail part hors du chemin du téléchargement.

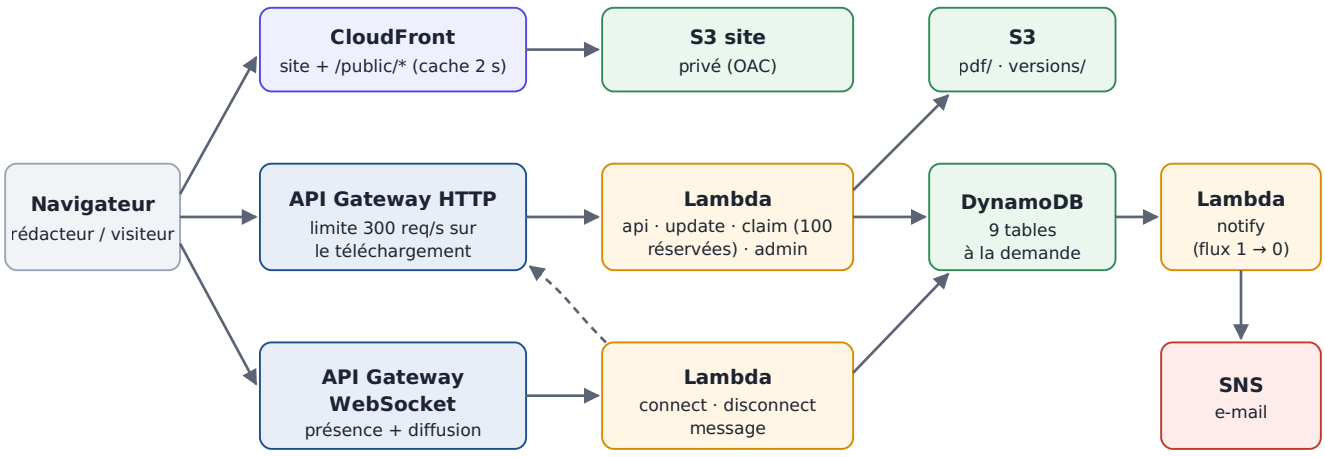


Schéma 1. L'architecture à construire, dans une seule région. Tout est « serverless » : aucune machine à dimensionner pour le pic, rien n'est facturé au repos. La flèche pointillée est la diffusion des modifications vers les navigateurs connectés.

Deux décisions de fond

Serverless plutôt qu'un serveur EC2. Un serveur Node.js aurait été plus simple à programmer (il garde le document en mémoire). Mais pour un pic d'une minute, il faudrait le dimensionner à l'avance, ajouter un équilibreur de charge et de l'auto-scaling (trop lent pour une minute), et le

payer en continu, ce qui va contre le plan gratuit (R-Compte). Lambda, API Gateway et DynamoDB montent en charge immédiatement, ne coûtent rien au repos, et permettent un rôle IAM par fonction (R-Droits).

AWS SAM plutôt que la console. Le sujet demande « la description de l'infrastructure » dans le dépôt et un README qui explique comment « déployer, tester et supprimer l'ensemble ». On décrit donc tout dans un fichier `template.yaml` (Infrastructure as Code, une bonne pratique : R-Pratiques). SAM est une extension de CloudFormation, plus courte pour décrire des fonctions et leurs déclencheurs, et un seul fichier YAML reste lisible par le jury. On ne clique jamais pour créer une ressource : une commande déploie, une commande supprime.

2. Ouvrir et préparer le compte AWS

2.1 Créer le compte

1. Sur `aws.amazon.com`, choisir **Créer un compte**. Il faut une adresse e-mail, un numéro de téléphone et une carte bancaire (vérification d'identité, aucun prélèvement sur le plan gratuit).
2. Au moment du choix de l'offre, prendre le **plan gratuit (Free plan)**. Ce plan **ne peut pas produire de facture** : la consommation est prélevée sur des crédits (100 $ à l'ouverture, plus des bonus d'activités). C'est exactement la consigne R-Compte, et une protection contre une erreur coûteuse.
3. Se connecter une première fois à la console avec ce compte.

2.2 Connaître sa région

Avec la nouvelle inscription d'AWS, le compte est rattaché à **une seule région**, déduite du pays de l'adresse. Pour la trouver : **AWS Settings** (`settings.aws.com`) › *Project* › *Overview* › *Additional Info*. Pour ce projet, c'était `eu-north-1`. La console n'a alors pas de sélecteur de région, et toute requête vers une autre région est refusée.

> **CONSIGNE · R-COMPTE « TOUTES VOS RESSOURCES SONT DANS UNE SEULE RÉGION »**
> Si votre compte est de l'ancien type, avec un sélecteur de région en haut à droite de la console, choisissez une région une fois pour toutes et utilisez-la partout : `samconfig.toml` , la commande `aws configure` et la variable `AWS_REGION` . CloudFront est un service mondial ; il ne crée aucune ressource dans une autre région, car on n'utilise ni certificat ni WAF (qui exigeraient `us-east-1`).

2.3 Vérifier le quota Lambda *avant* de construire

La contrainte C5 repose sur la **concurrence réservée** : on réserve 100 exécutions simultanées à la fonction de téléchargement. AWS exige qu'il reste au moins 100 exécutions non réservées sur le compte. Le compte doit donc avoir un quota d'au moins 200. Les comptes récents en ont parfois seulement 10. On vérifie à l'étape 4, une fois la CLI connectée :

Terminal

```
aws lambda get-account-settings --query AccountLimit.ConcurrentExecutions
400
# si la valeur est inférieure à 200 : demander une augmentation (réponse sous quelques heures à
quelques jours)
aws service-quotas request-service-quota-increase --service-code lambda \
  --quota-code L-B99A9384 --desired-value 1000
```

À FAIRE DÈS LE VENDREDI

Une augmentation de quota n'est pas immédiate. Si vous en avez besoin, demandez-la tout de suite. Sinon le premier déploiement échouera avec `Specified ReservedConcurrentExecutions ... decreases account's UnreservedConcurrentExecution below its minimum value of [100]`.

3. Équiper son poste

Cinq outils suffisent. Les commandes sont données pour Linux (x86_64) ; les équivalents macOS et Windows sont indiqués.

Outil	Rôle dans le projet	Pourquoi
AWS CLI v2 (≥ 2.32)	Parler à AWS depuis le terminal	Version 2.32 minimum pour `aws login`, qui évite de créer des clés d'accès (R-Clés).
AWS SAM CLI	Déployer `template.yaml`	Transforme le modèle en pile CloudFormation et envoie le code des fonctions.
Node.js 22 + npm	Construire backend et frontend, lancer les tests	Même version que l'environnement d'exécution Lambda (`nodejs22.x`).
Git	Le dépôt à rendre	Livrable n° 1.
openssl	Générer les secrets	Déjà présent sur Linux et macOS ; fourni avec Git pour Windows.

Terminal · AWS CLI v2

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip awscliv2.zip
sudo ./aws/install            # mise à jour d'une version existante : sudo ./aws/install --update
# macOS   : curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o AWSCLIV2.pkg && sudo installer -pkg
AWSCLIV2.pkg -target /
# Windows : msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi
```

Terminal · AWS SAM CLI

```
curl -L -o sam.zip https://github.com/aws/aws-sam-cli/releases/latest/download/aws-sam-cli-linux-
x86_64.zip
unzip sam.zip -d sam-installation
sudo ./sam-installation/install
# sans droits administrateur : ./sam-installation/install -i ~/.local/aws-sam-cli -b ~/.local/bin
# macOS : brew install aws-sam-cli   ·   Windows : installeur MSI sur la page des versions de aws-
sam-cli
```

Terminal · Node.js 22 (via nvm), Git

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
# fermer et rouvrir le terminal, puis :
nvm install 22
sudo apt install -y git unzip     # macOS : xcode-select --install · Windows : Git for Windows
```

Terminal · vérification

```
aws --version && sam --version && node --version && git --version && openssl version
aws-cli/2.38.4 ...   SAM CLI, version 1.170.0   v22.x   git version 2.x   OpenSSL 3.x
```

Pas besoin d'installer Docker : on n'utilise pas `sam build` (voir § 6.3). Pas besoin non plus d'installer k6, l'outil de test de charge, sur le poste : il sera installé sur la machine de test temporaire (§ 13).

4. Connecter la CLI à AWS, sans clé d'accès

La méthode habituelle (créer un utilisateur IAM et copier une « access key » dans `~/.aws/credentials`) laisse une clé permanente sur le disque. Une clé de ce genre finit tôt ou tard dans un dépôt. On ne crée donc **aucune clé** : `aws login` ouvre le navigateur, réutilise la session de la console et donne à la CLI des identifiants **temporaires**.

Terminal

```
aws login --region eu-north-1
aws configure set region eu-north-1
aws sts get-caller-identity
{ "Account": "896986967186",
  "Arn": "arn:aws:sts::896986967186:assumed-role/AccountFullAccessRole/8498e4d8-…" }
```

Le mot `assumed-role` dans l'ARN confirme qu'on travaille avec une session temporaire, et non avec un utilisateur muni de clés. La session dure au plus 12 heures ; quand la CLI répond *Your session has expired*, on relance `aws login`.

CONSIGNE · R-CLÉS « AUCUNE CLÉ D'ACCÈS AWS N'APPARAÎT DANS LE CODE OU DANS LE DÉPÔT »

Trois mécanismes couvrent toute l'application : `aws login` sur le poste ; un **rôle IAM** par fonction Lambda, dont les identifiants sont fournis automatiquement à l'exécution ; un rôle d'instance pour la machine de test. Aucun de ces identifiants ne peut se retrouver dans un fichier.

PIÈGE · « SOMETHING WENT WRONG » À LA CONNEXION

Avec la nouvelle expérience AWS, `aws login --region` doit recevoir la région du compte. Avec une autre région, par exemple `eu-west-3` , la page de connexion affiche seulement « Something went wrong ».

Vérifiez maintenant le quota Lambda (§ 2.3) et vos crédits :

Terminal

```
aws freetier get-account-plan-state --query
'{plan:accountPlanType,credits:accountPlanRemainingCredits}'
{ "plan": "FREE", "credits": { "amount": 119.98, "unit": "USD" } }
```

Notez ce chiffre et la date : c'est le premier point de la preuve des coûts (livrable n° 5).

5. Créer le dépôt et son squelette

Terminal

```
mkdir CollabDoc && cd CollabDoc && git init
mkdir -p backend/src/handlers backend/src/lib backend/scripts backend/test \
         scripts tests/load tests/convergence tests/results docs
```

L'arborescence finale, qui correspond aux livrables :

Arborescence

```
template.yaml    infrastructure (SAM)                       → description de l'infrastructure
samconfig.toml   paramètres de déploiement
backend/         fonctions Lambda + tests unitaires         → code de l'application
frontend/        application web (React + Vite)             → code de l'application
tests/           tests de bout en bout, de charge, résultats → tests, rapport de test
scripts/         bootstrap · deploy · destroy · loadtest    → déployer, tester, supprimer
docs/            conception · api · rapport-tests · couts   → livrables 2 à 5
README.md
```

5.1 Le fichier .gitignore

.gitignore

```
node_modules/
dist/
.aws-sam/
*.log
.env
.env.*
samconfig.local.toml
!tests/results/**/*.log
!tests/results/*.log
```

`dist/` et `.aws-sam/` sont des fichiers générés, qu'il ne faut pas versionner. `.env*` est exclu par précaution (R-Clés), même si le projet n'en utilise pas. Les deux dernières lignes font une exception pour les journaux de test, qui font partie des **résultats bruts** exigés dans le rapport de test.

5.2 Le fichier samconfig.toml

Ce fichier évite de retaper les options de déploiement. Il est lu par `sam deploy` et `sam delete`.

samconfig.toml

```
version = 0.1

[default.global.parameters]
stack_name = "collabdoc"          # nom de la pile CloudFormation
region = "eu-north-1"             # la seule région (R-Compte)

[default.deploy.parameters]
capabilities = "CAPABILITY_IAM"   # autorise SAM à créer les rôles IAM des fonctions
resolve_s3 = true                 # SAM crée lui-même le bucket où il dépose le code
s3_prefix = "collabdoc"
confirm_changeset = false         # pas de question interactive (script reproductible)
fail_on_empty_changeset = false   # redéployer sans changement n'est pas une erreur
tags = "project=\"collabdoc\""    # étiquette sur toutes les ressources : suivi des coûts
```

`CAPABILITY_IAM` est obligatoire : sans lui, CloudFormation refuse de créer des rôles, par sécurité. L'étiquette `project=collabdoc` permettra de compter les ressources, de filtrer les coûts (livrable n° 5) et de vérifier qu'elles ont toutes disparu à la fin.

6. Le backend : ce que le code doit faire

Le code des fonctions n'est pas détaillé ici. Ce qui suit est le **contrat** que votre code doit respecter, pour se brancher sur l'infrastructure du § 8 et pour tenir les contraintes. Le choix de TypeScript sur Node.js 22 vient de ce que le même langage sert au frontend, et que la bibliothèque Yjs (C2) est en JavaScript.

6.1 Initialiser le projet et installer les dépendances

Terminal · dans backend/

```
npm init -y && npm pkg set type=module
npm install @aws-sdk/client-dynamodb @aws-sdk/lib-dynamodb @aws-sdk/util-dynamodb \
            @aws-sdk/client-s3 @aws-sdk/s3-request-presigner @aws-sdk/client-sns \
            @aws-sdk/client-ssm @aws-sdk/client-apigatewaymanagementapi yjs pdf-lib
npm install -D typescript esbuild vitest @types/aws-lambda @types/node
npm pkg set scripts.build="node scripts/build.mjs" scripts.test="vitest run" scripts.typecheck="tsc
--noEmit"
```

Paquet	Usage
`@aws-sdk/*` (SDK v3)	DynamoDB, S3 et URL pré-signées, SNS (e-mail), SSM (secrets), API de gestion des connexions WebSocket. Le SDK lit tout seul les identifiants du rôle de la fonction : aucune clé dans le code.
`yjs`	Le CRDT (C2) : fusionner les modifications, compter les caractères insérés (C4).
`pdf-lib`	Générer le PDF en JavaScript pur, sans navigateur embarqué : le paquet de la fonction reste petit et son démarrage rapide (C5).
`esbuild`, `vitest`	Empaqueter chaque fonction ; tests unitaires.

6.2 Les neuf fonctions et leur responsabilité

Une fonction par fichier `backend/src/handlers/<nom>.ts`, qui exporte `handler`. Le code commun (accès DynamoDB, jetons, comptage, PDF) va dans `backend/src/lib/`.

Fichier	Déclencheur	Ce qu'il doit faire, et la garantie à respecter
`api`	HTTP : `/session`, `/me`, `/documents…` (membres, état, versions, liens)	Session : nom + e-mail → jeton signé HMAC avec le secret lu dans SSM. Créer et partager un document. `GET /state` : fusionner toutes les modifications stockées, en **lecture fortement cohérente** ; avec `?sv=`, ne renvoyer que ce qui manque au client. Versions : instantané texte dans S3 `versions/` ; une restauration est une nouvelle modification CRDT. Créer un lien : `remaining = maxDownloads`, puis abonner le créateur au sujet SNS avec une politique de filtre sur son adresse.
`update`	HTTP : `POST /documents/{id}/updates`	**C2 et C4.** Compter côté serveur les caractères insérés, puis exécuter une **transaction** : `Put` dans Updates avec `attribute_not_exists(updateId)` (un renvoi n'est jamais stocké deux fois) et `Update` du crédit avec la condition `used ≤ 10 000 − n`. Ensuite seulement : `ADD rev 1` sur le document (C3), réponse 200, diffusion aux connexions WebSocket du document. En cas de refus : 403 (crédit épuisé), avertissement d'abus ; au-delà de 20 avertissements en 10 minutes, blocage de 15 minutes (423).
`claim`	HTTP public : `POST /links/{id}/claim`, `GET /links/{id}`, `GET /public/links/{id}`	**C1, C3, C6.** Écriture conditionnelle `remaining − 1 si remaining > 0`. Échec → `410` épuisé. Succès → trace dans Claims, lecture cohérente de `rev`, PDF `pdf/<doc>/<rev>.pdf` lu dans S3 ou généré, puis URL pré-signée valable 60 s. Si le PDF échoue, rendre l'exemplaire (`+1`). Le statut public répond avec `Cache-Control: max-age=2`.
`admin`	HTTP : `GET /admin/state`	Vérifier l'en-tête `x-admin-token` (secret SSM). Lecture seule : documents ouverts (connexions actives), liens et exemplaires restants, personnes bloquées.
`wsConnect` / `wsDisconnect`	WebSocket `$connect` / `$disconnect`	Vérifier le jeton et l'accès au document, refuser une personne bloquée ; enregistrer ou retirer la connexion dans Connections ; annoncer la présence.
`wsMessage`	WebSocket `$default`	Relayer la présence (qui est là, où est son curseur) aux autres participants.
`notify`	Flux DynamoDB de Links	Publier un message SNS avec l'attribut `creator` ; le filtre SNS l'achemine au seul créateur du lien.
`snapshot`	Planificateur, toutes les 10 minutes	Enregistrer une version automatique de chaque document modifié depuis la dernière.

Les noms des tables, du bucket et des secrets ne sont jamais écrits en dur : le code les lit dans des **variables d'environnement** fournies par le modèle (§ 8.1) : `TABLE_DOCUMENTS`, `TABLE_MEMBERS`, `TABLE_UPDATES`, `TABLE_CREDITS`, `TABLE_CONNECTIONS`, `TABLE_LINKS`, `TABLE_CLAIMS`, `TABLE_VERSIONS`, `TABLE_ABUSE`, `DATA_BUCKET`, `WS_ENDPOINT`, `SESSION_SECRET_PARAM`, `ADMIN_TOKEN_PARAM`, `NOTIFICATION_TOPIC_ARN`, et `FRONTEND_URL` pour la seule fonction notify.

> **POURQUOI LE COMPTEUR DE LIEN N'EST PAS DANS UNE TRANSACTION, ALORS QUE LE CRÉDIT L'EST**
>
> Des transactions concurrentes sur le **même élément** se rejettent mutuellement (`TransactionConflict`). Pendant le pic, des milliers de visiteurs visent le même lien : une transaction produirait une foule d'échecs inutiles. L'écriture conditionnelle simple, elle, est appliquée à la suite sans conflit. Le crédit, au contraire, ne concerne qu'une personne, avec peu d'appareils : la transaction y est sans risque, et elle seule garantit le « tout ou rien » entre stocker et débiter.

6.3 Empaqueter les fonctions : backend/scripts/build.mjs

Ce script de 20 lignes passe chaque fichier de `src/handlers/` dans esbuild, avec les options `bundle: true`, `platform: 'node'`, `target: 'node22'`, `format: 'cjs'` et `minify: true`. Il produit `dist/<nom>/index.js`, un seul fichier par fonction qui contient tout son code et ses dépendances.

Pourquoi : SAM envoie le dossier `dist/<nom>` tel quel (`CodeUri` au § 8.8). On n'a donc besoin ni de `sam build` ni de Docker. Chaque fonction ne contient que ce qu'elle importe : le paquet est petit et le démarrage à froid court, ce qui compte pendant le pic. Le fichier `tsconfig.json` ne sert qu'à la vérification des types (`noEmit`, `strict`, `moduleResolution: bundler`).

6.4 Les tests unitaires

Dans `backend/test/*.test.ts` (Vitest) : comptage des caractères insérés, propriétés du CRDT (ordre indifférent, doublons sans effet), rattrapage par vecteur d'état, jetons signés, génération du PDF. Ils sont lancés automatiquement avant chaque déploiement (§ 10). Un test qui échoue arrête tout.

7. Le frontend : coder votre interface

L'interface est entièrement libre ; son apparence vous revient. On a utilisé React + Vite, avec CodeMirror 6 pour l'éditeur et l'extension `y-codemirror.next`, qui relie l'éditeur au CRDT et affiche les curseurs des autres.

Terminal · à la racine

```
npm create vite@latest frontend -- --template react-ts
cd frontend && npm install codemirror @codemirror/state @codemirror/view yjs y-codemirror.next y-protocols
```

Pour que les contraintes tiennent, l'interface doit en revanche respecter ces règles :

Comportement attendu	Raison
Lire les URL de l'API dans `window.COLLABDOC_CONFIG`, chargé par `<script src="/config.js">` dans `index.html`.	Les URL ne sont connues qu'après le déploiement : `deploy.sh` écrit ce fichier (§ 10). Le même code fonctionne pour tout compte, sans recompiler.
Regrouper les frappes (~80 ms), envoyer **une** modification à la fois, avec un identifiant unique, et la renvoyer avec le **même** identifiant en cas d'échec réseau.	C2 : rien ne se perd, et le serveur détecte les doublons.
Recharger l'état à chaque connexion et, toutes les 15 s, envoyer son vecteur d'état (`GET /state?sv=`).	C2 : toute diffusion WebSocket perdue est rattrapée ; tous convergent.
Page publique de téléchargement `#/d/<linkId>` : sur `429` / `503`, afficher « patientez » et réessayer avec un délai croissant aléatoire ; interroger le statut `/public/links/<id>` toutes les ~5 s.	C5 : chaque visiteur sait où il en est. C6 : il apprend l'épuisement sans même cliquer.
Page `#/admin` qui demande le jeton et affiche `GET /admin/state` toutes les 5 s.	« L'association doit pouvoir consulter à tout moment l'état de l'application. »

Pour développer en local contre l'application déployée : créer `frontend/public/config.js` avec les URL de l'API, puis `npm run dev`.

8. Décrire l'infrastructure : le fichier template.yaml

Voici le cœur du projet côté AWS. Le fichier fait environ 640 lignes ; on le décrit bloc par bloc, dans l'ordre où il est écrit, en donnant les réglages importants et la consigne qu'ils servent. CloudFormation crée les ressources dans l'ordre que leurs dépendances imposent, pas dans l'ordre du fichier.

8.1 En-tête, paramètres et réglages communs

template.yaml · début

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31      # active la syntaxe SAM

Parameters:
  SessionSecretParam: { Type: String, Default: /collabdoc/session-secret }
  AdminTokenParam:    { Type: String, Default: /collabdoc/admin-token }
  ClaimRateLimit:     { Type: Number, Default: 300 }   # téléchargements acceptés par seconde
  ClaimBurstLimit:    { Type: Number, Default: 300 }
  ClaimReservedConcurrency: { Type: Number, Default: 100 }

Globals:
  Function:
    Runtime: nodejs22.x
    Architectures: [arm64]       # processeurs Graviton : environ 20 % moins chers à performance
égale
    MemorySize: 512
    Timeout: 10
    Handler: index.handler       # le fichier index.js produit par build.mjs
    LoggingConfig: { LogGroup: !Ref LambdaLogGroup, LogFormat: Text }
    Tags: { project: collabdoc }
    Environment:
      Variables:
        TABLE_DOCUMENTS: !Ref DocumentsTable     # … une variable par table (liste au § 6.2)
        DATA_BUCKET: !Ref DataBucket
        WS_ENDPOINT: !Sub https://${WebSocketApi}.execute-api.${AWS::Region}.amazonaws.com/prod
        SESSION_SECRET_PARAM: !Ref SessionSecretParam
        ADMIN_TOKEN_PARAM: !Ref AdminTokenParam
        NOTIFICATION_TOPIC_ARN: !Ref LinkNotificationTopic
```

Les réglages du pic sont des **paramètres** : on peut les ajuster au déploiement sans modifier le modèle. Le bloc `Globals` évite de répéter neuf fois les mêmes réglages. Les variables d'environnement contiennent des **noms** de ressources et de paramètres, jamais des secrets.

8.2 Les journaux

template.yaml

```
LambdaLogGroup:
  Type: AWS::Logs::LogGroup
  Properties:
    LogGroupName: !Sub /collabdoc/${AWS::StackName}/lambda
    RetentionInDays: 14
```

Un seul groupe pour les neuf fonctions, ce qui simplifie l'enquête après un test. La rétention de 14 jours évite que les journaux s'accumulent et coûtent (R-Pratiques). Sans ce bloc, Lambda crée des groupes conservés indéfiniment.

8.3 Le bucket des fichiers téléchargés

template.yaml

```
DataBucket:
  Type: AWS::S3::Bucket
  Properties:
    PublicAccessBlockConfiguration:      # aucun accès public possible
      { BlockPublicAcls: true, BlockPublicPolicy: true, IgnorePublicAcls: true,
RestrictPublicBuckets: true }
    BucketEncryption:
      ServerSideEncryptionConfiguration: [{ ServerSideEncryptionByDefault: { SSEAlgorithm:
AES256 } }]
    LifecycleConfiguration:
      Rules: [{ Id: expire-old-pdf-renders, Status: Enabled, Prefix: pdf/, ExpirationInDays: 2 }]

DataBucketPolicy:                        # refuse toute requête non chiffrée (HTTP)
  Type: AWS::S3::BucketPolicy
  Properties:
    Bucket: !Ref DataBucket
    PolicyDocument:
      Statement:
        - { Sid: DenyInsecureTransport, Effect: Deny, Principal: '*', Action: 's3:*',
            Resource: [!GetAtt DataBucket.Arn, !Sub '${DataBucket.Arn}/*'],
            Condition: { Bool: { 'aws:SecureTransport': false } } }
```

CONSIGNE · R-S3 « LES FICHIERS À TÉLÉCHARGER SONT CONSERVÉS DANS S3 »

Les PDF sont rangés sous `pdf/<docId>/<rev>.pdf` et les versions sous `versions/`. Le bucket est privé : le visiteur reçoit une **URL pré-signée valable 60 s**, et le fichier transite directement de S3 vers son navigateur, sans passer par Lambda. Les PDF de plus de 2 jours sont supprimés seuls : ils peuvent toujours être régénérés à partir de l'état du document.

8.4 Les neuf tables DynamoDB

Toutes en `BillingMode: PAY_PER_REQUEST` (à la demande) : on ne prévoit aucune capacité, la table absorbe le pic (C5) et ne coûte rien au repos (R-Compte). Exemple complet, la table des liens :

template.yaml · LinksTable

```
LinksTable:
  Type: AWS::DynamoDB::Table
  Properties:
    BillingMode: PAY_PER_REQUEST
    AttributeDefinitions:
      - { AttributeName: linkId, AttributeType: S }
      - { AttributeName: docId,  AttributeType: S }
    KeySchema: [{ AttributeName: linkId, KeyType: HASH }]
    GlobalSecondaryIndexes:              # « les liens de ce document »
      - IndexName: byDoc
        KeySchema: [{ AttributeName: docId, KeyType: HASH }]
        Projection: { ProjectionType: ALL }
    StreamSpecification: { StreamViewType: NEW_AND_OLD_IMAGES }   # pour l'e-mail (C6)
    Tags: [{ Key: project, Value: collabdoc }]
```

Table	Clé (+ index)	Option	Rôle
Documents	`docId`		Titre, propriétaire, révision `rev` (C3).
Members	`email` + `docId` · index `byDoc`		Qui a accès à quel document, dans les deux sens.
Updates	`docId` + `updateId`		Les modifications Yjs acceptées : **la source de vérité du contenu** (C2).
Credits	`pk` = `email#jour`	TTL	Caractères utilisés ; un nouvel élément chaque jour, l'ancien expire (C4).
Connections	`connectionId` · index `byDoc` (clés seules)	TTL	Connexions WebSocket ouvertes ; l'index sert à la diffusion.
Links	`linkId` · index `byDoc`	Flux	`maxDownloads`, `remaining` (C1).
Claims	`linkId` + `claimId`		Un élément par exemplaire distribué : la trace qui sert de preuve pour C1.
Versions	`docId` + `versionId`		Métadonnées des versions (le texte est dans S3).
Abuse	`pk`	TTL	Avertissements par fenêtre et blocages, qui expirent seuls.

Le **TTL** (`TimeToLiveSpecification: { AttributeName: ttl, Enabled: true }`) supprime gratuitement les éléments dont la date `ttl` est passée. C'est ce qui « recharge » le crédit chaque jour et lève les blocages, sans tâche planifiée.

8.5 Le sujet SNS pour l'e-mail du dernier téléchargement

template.yaml

```
LinkNotificationTopic:
  Type: AWS::SNS::Topic
  Properties: { DisplayName: CollabDoc, Tags: [{ Key: project, Value: collabdoc }] }
```

Un seul sujet pour toute l'application. À la création d'un lien, la fonction api y abonne l'adresse du créateur avec une **politique de filtre** `{"creator": ["<son e-mail>"]}` : chacun ne reçoit que les messages de ses propres liens. Le choix de SNS plutôt que SES est expliqué au § 11.

8.6 L'API HTTP

template.yaml

```
HttpApi:
  Type: AWS::Serverless::HttpApi
  Properties:
    CorsConfiguration:
      AllowOrigins: ['*']
      AllowHeaders: [content-type, authorization, x-admin-token]
      AllowMethods: [GET, POST, OPTIONS]
      MaxAge: 600
    DefaultRouteSettings: { ThrottlingBurstLimit: 1000, ThrottlingRateLimit: 1000 }
    RouteSettings:
      'POST /links/{linkId}/claim':
        ThrottlingBurstLimit: !Ref ClaimBurstLimit     # 300
        ThrottlingRateLimit:  !Ref ClaimRateLimit      # 300 / s
```

On prend une **API HTTP** plutôt qu'une API REST : elle est environ trois fois moins chère, plus rapide, et suffit ici. Les routes ne sont pas déclarées dans ce bloc, mais dans chaque fonction (§ 8.8).

La **limite de débit** de la route de téléchargement est le premier rempart du pic (C5). 12 000 visiteurs en une minute font environ 200 requêtes par seconde en moyenne, beaucoup plus à la seconde de l'annonce. Au-delà de 300 par seconde, API Gateway répond lui-même `429` en quelques millisecondes, sans réveiller de fonction. La page affiche alors « patientez » et réessaie. Le visiteur reçoit une réponse au lieu d'une erreur, comme l'exige le sujet.

8.7 L'API WebSocket

Elle garde une connexion ouverte avec chaque rédacteur, pour lui pousser les modifications des autres et la présence (« qui est là, où il écrit »). SAM n'a pas de raccourci pour les WebSocket : on écrit les ressources CloudFormation de base. Il y a une API, un étage, et, pour chacune des trois routes, une intégration, une route et une permission.

template.yaml · extrait (route $connect, les deux autres sont identiques)

```
WebSocketApi:
  Type: AWS::ApiGatewayV2::Api
  Properties: { Name: !Sub '${AWS::StackName}-ws', ProtocolType: WEBSOCKET,
                RouteSelectionExpression: $request.body.action }
WebSocketStage:
  Type: AWS::ApiGatewayV2::Stage
  Properties:
    ApiId: !Ref WebSocketApi
    StageName: prod
    AutoDeploy: true
    DefaultRouteSettings: { ThrottlingBurstLimit: 5000, ThrottlingRateLimit: 5000 }
  DependsOn: [ConnectRoute, DisconnectRoute, DefaultRoute]

ConnectIntegration:                       # « cette route appelle cette fonction »
  Type: AWS::ApiGatewayV2::Integration
  Properties:
    ApiId: !Ref WebSocketApi
    IntegrationType: AWS_PROXY
    IntegrationUri: !Sub arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/$
{WsConnectFunction.Arn}/invocations
ConnectRoute:
  Type: AWS::ApiGatewayV2::Route
  Properties: { ApiId: !Ref WebSocketApi, RouteKey: $connect, Target: !Sub 'integrations/$
{ConnectIntegration}' }
ConnectPermission:                        # autorise CETTE API, et elle seule, à appeler la fonction
  Type: AWS::Lambda::Permission
  Properties:
    Action: lambda:InvokeFunction
    FunctionName: !Ref WsConnectFunction
    Principal: apigateway.amazonaws.com
    SourceArn: !Sub arn:aws:execute-api:${AWS::Region}:${AWS::AccountId}:${WebSocketApi}/*
```

Les trois routes sont `$connect` (wsConnect), `$disconnect` (wsDisconnect) et `$default` (wsMessage). Le `DependsOn` garantit que l'étage est créé une fois les routes en place. La limite de 5 000 messages par seconde a été **relevée après les tests** (§ 13.5).

> **POURQUOI LES MODIFICATIONS PASSENT PAR HTTP ET NON PAR LE WEBSOCKET**
>
> Une trame WebSocket d'API Gateway est limitée à 32 Kio, ce qu'un grand copier-coller dépasse, et il faudrait construire soi-même l'accusé de réception. En HTTP, la réponse 200 **est** l'accusé de réception : « acceptée » signifie « écrite dans DynamoDB » (C2). Le jury peut en plus tester l'API avec de simples requêtes HTTP. Le WebSocket ne sert qu'à **pousser** vers les navigateurs.

8.8 Les fonctions Lambda, chacune avec son propre rôle

Chaque fonction est une ressource `AWS::Serverless::Function`. Le champ `CodeUri` pointe vers le dossier produit par `build.mjs`, `Events` déclare ses déclencheurs, et `Policies` ses droits. SAM crée **un rôle IAM par fonction** à partir de `Policies`. Exemple, la fonction de téléchargement :

template.yaml · ClaimFunction

```
ClaimFunction:
  Type: AWS::Serverless::Function
  Properties:
    CodeUri: backend/dist/claim
    Timeout: 15
    MemorySize: 1024                                # plus de mémoire = plus de CPU pour générer
le PDF
    ReservedConcurrentExecutions: !Ref ClaimReservedConcurrency   # 100
    Policies:
      - Statement:
          - { Effect: Allow, Action: [dynamodb:GetItem, dynamodb:UpdateItem], Resource: !GetAtt
LinksTable.Arn }
          - { Effect: Allow, Action: [dynamodb:PutItem], Resource: !GetAtt ClaimsTable.Arn }
          - { Effect: Allow, Action: [dynamodb:GetItem], Resource: !GetAtt DocumentsTable.Arn }
          - { Effect: Allow, Action: [dynamodb:Query],   Resource: !GetAtt UpdatesTable.Arn }
          - { Effect: Allow, Action: [s3:GetObject, s3:PutObject], Resource: !Sub '$
{DataBucket.Arn}/pdf/*' }
    Events:
      Claim:        { Type: HttpApi, Properties: { ApiId: !Ref HttpApi, Method: POST, Path: '/
links/{linkId}/claim' } }
      Status:       { Type: HttpApi, Properties: { ApiId: !Ref HttpApi, Method: GET,  Path: '/
links/{linkId}' } }
      PublicStatus: { Type: HttpApi, Properties: { ApiId: !Ref HttpApi, Method: GET,  Path: '/
public/links/{linkId}' } }
```

CONSIGNE · C5 « PENDANT QUE 300 RÉDACTEURS CONTINUENT D'ÉCRIRE »

Le compte dispose de 400 exécutions Lambda simultanées, à partager entre toutes les fonctions. `ReservedConcurrentExecutions: 100` garantit 100 places à la fonction de téléchargement, mais surtout l'**empêche d'en prendre plus**. Même si 15 000 visiteurs arrivent ensemble, il reste 300 places pour la fonction d'édition : les rédacteurs ne sentent pas le pic. Les visiteurs en trop reçoivent « patientez » (429 ou 503).

Les droits de chaque fonction, tels qu'écrits dans le modèle :

Fonction	Déclencheurs	Droits accordés (et rien d'autre)
api Timeout 20	13 routes HTTP : `POST /session`, `GET /me`, `GET|POST /documents`, `GET /documents/{docId}`, `…/members`, `…/state`, `…/versions` (liste, création, lecture, `…/restore`), `…/links` (liste, création)	Lire le secret de session ; CRUD sur Documents, Members, Updates, Versions, Links ; lecture seule sur Credits, Abuse ; `Query` sur l'index `Connections/byDoc` et `DeleteItem` sur Connections ; S3 `versions/*` ; `sns:Subscribe` et `sns:ListSubscriptionsByTopic` sur le sujet ; envoyer aux connexions WebSocket.
update	`POST /documents/{docId}/updates`	Lire le secret de session ; `PutItem` sur Updates ; `UpdateItem` sur Documents, Credits, Abuse ; `GetItem` sur Members, Credits, Abuse ; `PutItem` sur Abuse ; l'index Connections, `DeleteItem` sur Connections ; envoyer aux connexions WebSocket.
claim	3 routes publiques (ci-dessus)	Links (lire, décrémenter), Claims (écrire), Documents (lire), Updates (lire), S3 `pdf/*`. Rien sur le crédit, SSM ou SNS.
admin	`GET /admin/state`	Lire le jeton admin ; `Scan` sur Connections, Links, Abuse ; `BatchGetItem` sur Documents. Lecture seule.
wsConnect	`$connect`	Lire le secret de session ; `GetItem` sur Members, Abuse ; écrire et supprimer dans Connections, l'index ; envoyer aux connexions WebSocket.
wsDisconnect	`$disconnect`	`DeleteItem` sur Connections, l'index ; envoyer aux connexions WebSocket.
wsMessage	`$default`	`GetItem` et `DeleteItem` sur Connections, l'index ; envoyer aux connexions WebSocket.
notify	Flux de Links (§ 8.9)	`sns:Publish` sur le seul sujet du projet.
snapshot Timeout 120	Toutes les 10 minutes (§ 8.9)	Documents (`Scan` , lire, mettre à jour), Updates (lire), Versions (écrire), S3 `versions/*` (écrire).

« Envoyer aux connexions WebSocket » s'écrit `execute-api:ManageConnections` sur la seule ressource `…/${WebSocketApi}/prod/POST/@connections/*`. Le droit de lire un secret est limité au seul paramètre concerné : `arn:aws:ssm:${AWS::Region}:${AWS::AccountId}:parameter${SessionSecretParam}`.

CONSIGNE · R-DROITS « CHAQUE COMPOSANT N'A QUE LES DROITS DONT IL A BESOIN »

On écrit les droits action par action et ressource par ressource, sans `*` et sans politique d'administration. Les modèles SAM `DynamoDBCrudPolicy` et `DynamoDBReadPolicy`, utilisés pour la fonction api, sont limités chacun à une table. La fonction de téléchargement, exposée à des inconnus, ne peut toucher ni aux crédits, ni aux secrets, ni à l'envoi d'e-mails. Le § 12.3 montre comment le **prouver**.

8.9 Deux déclencheurs particuliers : le flux DynamoDB et le planificateur

template.yaml · NotifyFunction et SnapshotFunction (extraits)

```
NotifyFunction:
  Properties:
    CodeUri: backend/dist/notify
    Environment: { Variables: { FRONTEND_URL: !Sub 'https://${Distribution.DomainName}' } }
    Policies: [{ Statement: [{ Effect: Allow, Action: sns:Publish, Resource: !Ref
LinkNotificationTopic }] }]
    Events:
      LastCopy:
        Type: DynamoDB
        Properties:
          Stream: !GetAtt LinksTable.StreamArn
          StartingPosition: LATEST
          BatchSize: 10
          MaximumRetryAttempts: 5
          FilterCriteria:
            Filters:
              - Pattern: '{"eventName":["MODIFY"],"dynamodb":{"NewImage":{"remaining":{"N":
["0"]}},"OldImage":{"remaining":{"N":["1"]}}}}'

SnapshotFunction:
  Properties:
    CodeUri: backend/dist/snapshot
    Timeout: 120
    Events:
      Every10Min: { Type: ScheduleV2, Properties: { ScheduleExpression: rate(10 minutes) } }
```

Le **filtre** ne laisse passer qu'un seul événement par lien : le passage de `remaining` de 1 à 0. Pendant le pic, les 499 autres décréments n'invoquent même pas la fonction. L'e-mail est hors du chemin du téléchargement : s'il échoue, aucun visiteur n'est bloqué, et Lambda réessaie jusqu'à 5 fois (C6, « son créateur en est prévenu par e-mail »). `ScheduleV2` crée une planification EventBridge Scheduler, qui produit les versions automatiques (« consulter les versions précédentes »).

PIÈGE · « CIRCULAR DEPENDENCY BETWEEN RESOURCES »

Une première version donnait l'URL CloudFront à *toutes* les fonctions, via `Globals`. Or CloudFront dépend de l'API, qui dépend des fonctions : CloudFormation refuse le cycle. La correction : ne donner `FRONTEND_URL` qu'à notify, qui ne dépend pas de l'API.

8.10 Le site : bucket privé, Origin Access Control et CloudFront

template.yaml · extraits

```
FrontendBucket:                 # même blocage de l'accès public que DataBucket
FrontendOAC:
  Type: AWS::CloudFront::OriginAccessControl
  Properties:
    OriginAccessControlConfig: { Name: !Sub '${AWS::StackName}-oac', OriginAccessControlOriginType:
s3,
                                 SigningBehavior: always, SigningProtocol: sigv4 }
FrontendBucketPolicy:           # seule CETTE distribution peut lire le bucket
  Statement:
    - Effect: Allow
      Principal: { Service: cloudfront.amazonaws.com }
      Action: s3:GetObject
      Resource: !Sub '${FrontendBucket.Arn}/*'
      Condition: { StringEquals: { 'AWS:SourceArn': !Sub 'arn:aws:cloudfront::$
{AWS::AccountId}:distribution/${Distribution}' } }

Distribution:
  Type: AWS::CloudFront::Distribution
  Properties:
    DistributionConfig:
      Enabled: true
      DefaultRootObject: index.html
      PriceClass: PriceClass_100          # points de présence Europe + Amérique du Nord : le moins
cher
      HttpVersion: http2and3
      Origins:
        - { Id: frontend, DomainName: !GetAtt FrontendBucket.RegionalDomainName,
            OriginAccessControlId: !GetAtt FrontendOAC.Id, S3OriginConfig: { OriginAccessIdentity:
'' } }
        - { Id: api, DomainName: !Sub '${HttpApi}.execute-api.${AWS::Region}.amazonaws.com',
            CustomOriginConfig: { OriginProtocolPolicy: https-only, OriginSSLProtocols:
[TLSv1.2] } }
      DefaultCacheBehavior:               # le site
        { TargetOriginId: frontend, ViewerProtocolPolicy: redirect-to-https, Compress: true,
          CachePolicyId: 658327ea-f89d-4fab-a63d-7e88639e58f6 }   # politique gérée «
CachingOptimized »
      CacheBehaviors:
        - PathPattern: /public/*          # le statut des liens, depuis l'API
          TargetOriginId: api
          ViewerProtocolPolicy: https-only
          AllowedMethods: [GET, HEAD]
          CachePolicyId: 658327ea-f89d-4fab-a63d-7e88639e58f6     # respecte le max-age=2 de la
réponse
          Compress: true
```

Pourquoi CloudFront : 15 000 personnes ouvrent la page en une minute. Servies par CloudFront, ces pages ne touchent jamais l'API (C5), et le volume tient dans le niveau gratuit. Le bucket reste privé, sans « hébergement de site statique » public : seul CloudFront le lit, grâce à l'**Origin Access Control** (R-Pratiques). Le second comportement, `/public/*`, est le levier de C6. Les pages ouvertes interrogent le statut du lien toutes les ~5 s ; grâce au cache de 2 s, 15 000 pages ne coûtent à l'API que quelques requêtes par seconde, et l'épuisement s'affiche partout en quelques secondes.

8.11 Les sorties

template.yaml · fin

```
Outputs:
  ApiUrl:             { Value: !Sub 'https://${HttpApi}.execute-api.${AWS::Region}.amazonaws.com' }
  WebSocketUrl:       { Value: !Sub 'wss://${WebSocketApi}.execute-api.$
{AWS::Region}.amazonaws.com/prod' }
  FrontendUrl:        { Value: !Sub 'https://${Distribution.DomainName}' }
  FrontendBucketName: { Value: !Ref FrontendBucket }
  DistributionId:     { Value: !Ref Distribution }
  DataBucketName:     { Value: !Ref DataBucket }
  LogGroup:           { Value: !Ref LambdaLogGroup }
```

Les noms réels des ressources sont générés par AWS. Les sorties sont le seul endroit où les scripts vont les chercher. Ainsi, rien n'est écrit en dur, et le projet se déploie à l'identique sur n'importe quel compte.

8.12 Valider le modèle avant de déployer

Terminal

```
sam validate --lint --region eu-north-1
template.yaml is a valid SAM Template
```

`--lint` lance cfn-lint, qui détecte les propriétés mal orthographiées, les références vers des ressources inexistantes et les valeurs interdites, avant qu'un déploiement n'échoue au bout de cinq minutes.

9. Créer les secrets sans les écrire nulle part

L'application a deux secrets : la clé qui signe les sessions des rédacteurs et le jeton du tableau de bord de l'association. Le script `scripts/bootstrap.sh`, à lancer une seule fois, les génère directement dans **SSM Parameter Store**, chiffrés par KMS (type `SecureString`). Ils ne passent jamais par un fichier.

scripts/bootstrap.sh · le cœur du script

```
REGION="${AWS_REGION:-eu-north-1}"
create_if_missing() {
  if aws ssm get-parameter --region "$REGION" --name "$1" >/dev/null 2>&1; then echo "exists:  $1"
  else
    aws ssm put-parameter --region "$REGION" --name "$1" --type SecureString \
      --value "$(openssl rand -base64 32 | tr -d '=+/')" --tags Key=project,Value=collabdoc >/dev/
null
    echo "created: $1"
  fi
}
create_if_missing /collabdoc/session-secret
create_if_missing /collabdoc/admin-token
```

Terminal

```
chmod +x scripts/*.sh
./scripts/bootstrap.sh
created: /collabdoc/session-secret
created: /collabdoc/admin-token
```

Le script ne recrée pas un secret existant (*idempotent*) : on peut le relancer sans invalider les sessions en cours. Les noms de ces paramètres sont ceux du § 8.1, et seules les fonctions qui en ont besoin ont le droit `ssm:GetParameter` dessus. Avant chaque envoi vers GitHub, une vérification rapide :

Terminal

```
git ls-files | xargs grep -lE "AKIA[0-9A-Z]{16}|aws_secret_access_key" || echo "aucune clé"
aucune clé
```

10. Déployer d'une seule commande

Le script `scripts/deploy.sh` enchaîne tout. Il sert au développeur comme au correcteur qui voudra redéployer le projet.

scripts/deploy.sh · les étapes

```
set -euo pipefail                                          # la moindre erreur arrête tout
(cd backend && npm ci && npm test && npm run build)        # 1. tests unitaires puis empaquetage
sam deploy --region "$REGION"                              # 2. infrastructure (lit samconfig.toml)
out() { aws cloudformation describe-stacks --stack-name collabdoc \
          --query "Stacks[0].Outputs[?OutputKey=='$1'].OutputValue" --output text; }
API_URL=$(out ApiUrl); WS_URL=$(out WebSocketUrl); BUCKET=$(out FrontendBucketName); DIST=$(out
DistributionId)
(cd frontend && npm ci && npm run build)                   # 3. site
printf "window.COLLABDOC_CONFIG = { apiUrl: '%s', wsUrl: '%s' }\n" "$API_URL" "$WS_URL" > frontend/
dist/config.js
aws s3 sync frontend/dist "s3://$BUCKET" --delete --exclude index.html --exclude config.js \
    --cache-control "public, max-age=31536000, immutable"  # 4. fichiers à nom unique : cache 1 an
aws s3 cp frontend/dist/index.html "s3://$BUCKET/index.html" --cache-control "no-cache"
aws s3 cp frontend/dist/config.js  "s3://$BUCKET/config.js"  --cache-control "no-cache" --content-
type application/javascript
aws cloudfront create-invalidation --distribution-id "$DIST" --paths "/index.html" "/config.js"
```

Terminal

```
./scripts/deploy.sh
Successfully created/updated stack - collabdoc in eu-north-1
Deployed.
  Frontend : https://d2cibe3b2w1o4h.cloudfront.net
  API      : https://yoi6qc6jul.execute-api.eu-north-1.amazonaws.com
  WebSocket: wss://ve8vn41ool.execute-api.eu-north-1.amazonaws.com/prod
  Admin    : https://d2cibe3b2w1o4h.cloudfront.net/#/admin
```

Le premier déploiement prend une dizaine de minutes, surtout pour CloudFront. Les suivants ne mettent à jour que ce qui a changé. Les fichiers produits par Vite portent une empreinte dans leur nom : on peut les mettre en cache un an. `index.html` et `config.js` sont en `no-cache`, et on invalide leur copie dans CloudFront pour qu'une nouvelle version soit visible tout de suite.

Dans la console, **CloudFormation › Stacks › collabdoc › Resources** liste tout ce qui a été créé. Pour compter les ressources étiquetées et récupérer le jeton du tableau de bord :

Terminal

```
aws resourcegroupstaggingapi get-resources --tag-filters Key=project,Values=collabdoc \
  --query 'length(ResourceTagMappingList)'
30
aws ssm get-parameter --name /collabdoc/admin-token --with-decryption --query Parameter.Value --
output text
```

11. Samedi : essayer l'application, et l'histoire de l'e-mail

On ouvre l'URL du site, on saisit un nom et un e-mail, on crée un document, on ouvre le même document dans un second navigateur avec un autre nom : les curseurs et les modifications apparaissent des deux côtés. On crée un lien de 2 exemplaires, on ouvre `#/d/<linkId>`, on télécharge deux fois, la troisième tentative affiche « épuisé ».

À la création de son **premier** lien, le rédacteur reçoit un e-mail *« AWS Notification - Subscription Confirmation »* envoyé par `no-reply@sns.amazonaws.com`. Il clique une fois sur *Confirm subscription*. Il sera ensuite prévenu à chaque dernier téléchargement de ses liens. Le README doit le signaler au jury.

PIÈGE · POURQUOI SNS ET PAS SES (AMAZON SIMPLE EMAIL SERVICE)

La première version envoyait l'e-mail avec SES, depuis l'adresse Gmail du développeur, vérifiée dans SES. En démonstration, aucun e-mail n'est arrivé. Les journaux CloudWatch (**CloudWatch › Log groups › /collabdoc/collabdoc/lambda**) ont montré deux causes :

- un envoi accepté par SES, puis filtré par Gmail : un message « @gmail.com » qui ne sort pas des serveurs de Google échoue au contrôle anti-usurpation **DMARC** ;
- un envoi refusé : un compte SES neuf est en **sandbox** et n'écrit qu'à des adresses vérifiées une par une.

Sans nom de domaine à soi (nécessaire pour configurer DKIM et SPF) et sans le délai d'une demande de sortie de sandbox, SES ne convient pas pour un week-end. SNS écrit à n'importe quelle adresse, depuis une adresse d'AWS. La contrepartie, une confirmation unique par destinataire, est une règle anti-spam d'AWS. **Faites directement le choix de SNS.**

12. Prouver chaque contrainte, dans le code et sur AWS

12.1 Les tests de bout en bout

Ils sont écrits en Node.js, dans `tests/`, avec les paquets `ws` et `yjs`. Ils n'utilisent **que l'API publique**, comme le ferait le jury. Chacun écrit ses résultats bruts en JSON dans `tests/results/` et affiche PASS ou FAIL.

Terminal

```
cd backend && npm test && cd ..                                  # tests unitaires
cd tests && npm ci
FRONT_URL=<FrontendUrl> node smoke.mjs                             # parcours complet
WRITERS=50 EDITS=20 node convergence/convergence.mjs               # C2 : 50 rédacteurs à la
position 0
MAX=100 CLICKS=3000 node load/claim-race.mjs                       # C1 : 3 000 clics
simultanés, 100 exemplaires
DEVICES=10 node load/credit-multidevice.mjs                        # C4 : 10 appareils, puis
acharnement
```

Test	Comment il prouve la contrainte
convergence (C2)	Chaque modification insère un marqueur unique. On vérifie que chaque marqueur dont l'envoi a reçu 200 apparaît **exactement une fois** dans le document du serveur, puis que la copie de chaque participant est identique.
claim-race (C1)	Tous les clics partent au même instant, sans nouvel essai. On compte les 200, on relit `remaining` et on vérifie que les numéros d'exemplaire sont tous distincts (1..N).
credit-multidevice (C4)	Le même e-mail sur 10 sessions qui écrivent en parallèle : exactement 10 000 caractères acceptés, puis un refus, puis le blocage après 20 refus.

12.2 Relire la preuve directement dans AWS

Le jury croira davantage ce qu'il voit dans AWS que ce qu'affiche un script. Les noms réels des tables se trouvent dans **CloudFormation › collabdoc › Resources**.

Terminal · C1 : lien de 2 exemplaires, 3 demandes

```
for i in 1 2 3; do curl -s -o /dev/null -w "%{http_code}\n" -X POST "$API/links/$LINK/claim"; done
200
200
410
aws dynamodb get-item --table-name <LinksTable> --key '{"linkId":{"S":"'$LINK'"}}' \
  --query 'Item.{max:maxDownloads.N,restants:remaining.N}'
{ "max": "2", "restants": "0" }
aws dynamodb query --table-name <ClaimsTable> --key-condition-expression "linkId = :l" \
  --expression-attribute-values '{":l":{"S":"'$LINK'"}}' --select COUNT --query Count
2
```

Terminal · C3 et C4

```
aws s3 ls s3://<DataBucket>/pdf/$DOC/             # un PDF par révision : même lien, document à
jour
2026-10-25 04:47:54   1064 1.pdf
2026-10-25 04:47:58   1088 2.pdf
aws dynamodb get-item --table-name <CreditsTable> --key '{"pk":{"S":"<email>#2026-10-24"}}' --query
'Item.used.N'
"10000"
```

Terminal · C5 et C6 : les protections sont bien en place

```
aws apigatewayv2 get-stage --api-id <HttpApi> --stage-name '$default' --query RouteSettings
{ "POST /links/{linkId}/claim": { "ThrottlingBurstLimit": 300, "ThrottlingRateLimit": 300.0 } }
aws lambda get-function-concurrency --function-name <ClaimFunction>
{ "ReservedConcurrentExecutions": 100 }
aws lambda list-event-source-mappings --function-name <NotifyFunction> \
  --query 'EventSourceMappings[0].FilterCriteria.Filters[0].Pattern'
```

12.3 Prouver le moindre privilège avec le simulateur IAM

Terminal

```
aws iam simulate-principal-policy --policy-source-arn <rôle de ClaimFunction> \
  --action-names dynamodb:UpdateItem --resource-arns <ARN de CreditsTable> \
  --context-entries ContextKeyName=aws:RequestedRegion,ContextKeyValues=eu-
north-1,ContextKeyType=string \
  --query 'EvaluationResults[0].EvalDecision'
"implicitDeny"
```

La fonction de téléchargement est autorisée à décrémenter Links (`allowed`), mais pas à modifier un crédit, supprimer une table ou publier sur SNS (`implicitDeny`). Le paramètre `--context-entries` est indispensable sur un compte « projet » : sans lui, le simulateur ne connaît pas la région de la requête, applique la règle d'organisation qui refuse les autres régions, et répond `explicitDeny` à tout.

13. Dimanche : le test de pic (C5 et C6)

13.1 Pourquoi une machine de test dans la région

Les premiers tests, lancés depuis un ordinateur personnel, échouaient au-delà d'environ 300 connexions simultanées (`UND_ERR_CONNECT_TIMEOUT`), alors que CloudWatch ne montrait aucune erreur côté AWS. On mesurait la connexion internet, pas l'application. Pour simuler 15 000 visiteurs, on crée donc une machine EC2 **temporaire** dans la même région, elle aussi décrite par un modèle : `tests/load/loadgen.yaml`.

13.2 Le modèle de la machine : tests/load/loadgen.yaml

Ressource	Réglage	Justification
ArtifactsBucket	bucket privé	Faire passer les scripts de test vers la machine et rapatrier les résultats.
Role + InstanceProfile	politique gérée `AmazonSSMManagedInstanceCore` + lecture/écriture sur ce seul bucket	Piloter la machine par SSM, sans clé (R-Clés, R-Droits).
SecurityGroup	**aucune règle d'entrée**	Pas de SSH, pas de port ouvert : rien à attaquer.
Instance	`m7i-flex.large` (2 vCPU, 8 Gio), Amazon Linux 2023 (image lue dans SSM), `HttpTokens: required` (IMDSv2), disque chiffré, `InstanceInitiatedShutdownBehavior: terminate`	Le plus gros type autorisé par le plan gratuit (R-Compte). Bonnes pratiques de sécurité EC2.
UserData	`shutdown -h +240`, installation de Node.js et de k6, hausse des limites de fichiers et de ports	La machine se supprime d'elle-même au bout de 4 heures, même si on oublie de le faire. Les limites système permettent des milliers de connexions.

PIÈGE · « THE SPECIFIED INSTANCE TYPE IS NOT ELIGIBLE FOR FREE TIER »

Le plan gratuit n'accepte que certains types d'instance. Listez-les avant de choisir :

```
aws ec2 describe-instance-types --filters Name=free-tier-eligible,Values=true \
  --query 'InstanceTypes[].[InstanceType,VCpuInfo.DefaultVCpus,MemoryInfo.SizeInMiB]' --output
text
```

Résultat : `t3.micro`, `t4g.micro`, `t3.small`, `t4g.small`, `c7i-flex.large`, `m7i-flex.large`. On prend le dernier.

13.3 Piloter la machine : scripts/loadtest.sh

Terminal

```
./scripts/loadtest.sh up                        # aws cloudformation deploy de loadgen.yaml (~3
min), attend l'agent SSM
./scripts/loadtest.sh run 'bash load/peak.sh'   # envoie tests/ par S3, exécute par SSM Run Command
(~8 min)
./scripts/loadtest.sh pull                      # rapatrie tests/results/
tests/load/cloudwatch-metrics.sh <début UTC> <fin UTC> > tests/results/cloudwatch-<run>.json
./scripts/loadtest.sh down                      # vide le bucket et supprime la pile collabdoc-
loadgen
```

`run` archive le dossier `tests/`, le dépose dans le bucket, puis lance `aws ssm send-command --document-name AWS-RunShellScript` sur la machine avec les URL de l'application, et attend la fin en interrogeant `get-command-invocation`. Tout passe par les API d'AWS : aucune connexion directe à la machine. Pour ces tests, k6 est le bon outil : il simule des milliers de visiteurs depuis une seule machine.

13.4 Le scénario et les résultats

`tests/load/peak.sh` démarre 300 rédacteurs qui écrivent sans interruption dans 30 documents pendant 7 minutes (`writers.mjs`). Il crée ensuite un lien de 500 exemplaires et lance k6 (`spike.js`) : 1 500 visiteurs à la seconde même de l'annonce, 12 000 dans la première minute (80 %), puis le reste sur 4 minutes. Chaque visiteur fait comme la page : il lit le statut, demande un exemplaire, attend et réessaie sur « patientez », jusqu'à « téléchargé » ou « épuisé ». `cloudwatch-metrics.sh` relève ensuite la vue côté serveur : invocations, erreurs, limitations, concurrence et latences.

Mesure (pic final)	Valeur	Ce que cela prouve
Téléchargements réussis / exemplaires	**500 / 500**	C1 sous la pire charge
Modifications acceptées et retrouvées / envoyées	**79 334 / 79 334**, 0 en double	C2 sous charge
Visiteurs sans réponse finale · réponses inattendues	0 · 0	C5 : « sans casser, ni jeter »
Erreurs Lambda · requêtes DynamoDB limitées	0 · 0	Rien n'a cassé côté AWS
Concurrence max : téléchargement / édition	100 (plafond) / 75	Le cloisonnement a protégé les rédacteurs
Latence d'une modification, p99	221 ms	Les rédacteurs n'ont rien senti
Délai de la réponse « épuisé », p50	44 ms	C6 : on l'apprend vite
Invocations de la fonction e-mail	1	Un seul e-mail au créateur

13.5 Ce que les tests ont fait corriger

- **Connexions supprimées pendant leur ouverture.** API Gateway répond « Gone » pour une connexion WebSocket encore en cours d'ouverture, et le code la supprimait : la moitié des rédacteurs ne recevaient plus rien en direct. Correction : ne supprimer une connexion « Gone » que si elle a plus de 30 s.
- **38 632 envois refusés (`TooManyRequestsException`)** avec 50 rédacteurs dans un même document. Corrections : la limite de l'étage WebSocket relevée à 5 000 messages par seconde (§ 8.7), les envois refusés réessayés, et surtout la resynchronisation par vecteur d'état toutes les 15 s côté navigateur (§ 7).

Dans les deux cas, aucune modification *acceptée* n'était perdue : DynamoDB contenait tout, et seul l'affichage en direct était en retard. Gardez les résultats d'avant correction dans `tests/results/archive/` : ils montrent au jury que les tests ont servi.

À LA FIN DE CHAQUE SÉANCE

```
aws ec2 describe-instances --filters Name=instance-state-name,Values=pending,running --query
'length(Reservations)'
0
```

14. Prouver ce que le projet a coûté

Le livrable n° 5 demande une *preuve*. On en construit trois : les prix officiels, les volumes mesurés et les relevés du compte.

Terminal

```
# 1. Prix officiels de la région (l'API de tarification n'existe qu'en us-east-1 ; c'est une
lecture, pas une ressource)
aws pricing get-products --region us-east-1 --service-code AmazonApiGateway \
  --filters Type=TERM_MATCH,Field=location,Value="EU (Stockholm)"
# 2. Crédits restants, à relever avant, pendant et après
aws freetier get-account-plan-state
# 3. Coûts réels par service et par jour (publiés avec environ 24 h de retard)
aws ce get-cost-and-usage --region us-east-1 --time-period Start=2026-10-24,End=2026-10-27 \
  --granularity DAILY --metrics UnblendedCost --group-by Type=DIMENSION,Key=SERVICE
```

Poste	Volume mesuré	Coût
Un pic complet (WebSocket, HTTP, DynamoDB)	786 447 messages, 116 368 requêtes, ≈ 0,75 M d'unités DynamoDB	≈ 1,30 $
Machine de test m7i-flex.large	≈ 1 h à 0,10175 $/h	≈ 0,10 $
Lambda, CloudFront, S3, SNS, SSM	dans les niveaux toujours gratuits	≈ 0 $
Projet entier (3 pics, tests, déploiements)		**≈ 4,50 $ de crédits, 0 $ payé**

Complétez par des captures de **Billing and Cost Management › Cost Explorer**, filtrées sur la région et l'étiquette `project`, et regroupées par service. Au repos, l'application ne coûte rien : aucune machine n'est allumée.

15. Lundi matin : rendre, puis tout supprimer

15.1 Vérifier les livrables

Le sujet demande	Où c'est dans le dépôt
Code, infrastructure, tests, README	`backend/`, `frontend/`, `template.yaml` + `samconfig.toml`, `tests/`, `README.md` (sections Déployer, Tester, Supprimer)
Dossier de conception	`docs/conception.md` : le tableau du § 1, les alternatives écartées, le modèle de données, la sécurité, les limites
Rapport de test (méthode, objectifs, résultats bruts, analyse, preuve de C1 et C2)	`docs/rapport-tests.md` + `tests/results/*.json`
Description de l'API	`docs/api.md` : chaque route, ses paramètres, ses réponses (200, 403, 410, 423, 429, 503) et les messages WebSocket
Preuve des coûts	`docs/couts.md` + captures Cost Explorer

Terminal

```
git add -A && git commit -m "Projet CollabDoc"
git remote add origin git@github.com:<compte>/CollabDoc.git && git push -u origin main
```

15.2 Tout supprimer, après la soutenance

> **PAS AVANT LA PRÉSENTATION**
>
> La démonstration a besoin de l'application en ligne. Au repos, elle ne coûte rien.

scripts/destroy.sh · le cœur du script

```
for b in "$(out FrontendBucketName)" "$(out DataBucketName)"; do
  aws s3 rm "s3://$b" --recursive                # CloudFormation refuse de supprimer un bucket
non vide
done
sam delete --stack-name collabdoc --no-prompts    # tables, fonctions, API, CloudFront, SNS et
abonnements
aws ssm delete-parameters --names /collabdoc/session-secret /collabdoc/admin-token
```

Terminal

```
./scripts/destroy.sh
aws resourcegroupstaggingapi get-resources --tag-filters Key=project,Values=collabdoc \
  --query 'length(ResourceTagMappingList)'
0
```

Il reste le petit bucket où SAM dépose le code, dans la pile `aws-sam-cli-managed-default`, qui ne coûte presque rien. Pour le supprimer aussi : le vider (y compris ses anciennes versions) depuis la console S3, puis `aws cloudformation delete-stack --stack-name aws-sam-cli-managed-default`.

Annexe A · Toutes les commandes, dans l'ordre

Du compte vide à l'application en ligne

```
# Vendredi : poste et compte
aws --version && sam --version && node --version            # outils installés (§ 3)
aws login --region eu-north-1 && aws configure set region eu-north-1
aws sts get-caller-identity
aws lambda get-account-settings --query AccountLimit.ConcurrentExecutions   # ≥ 200, sinon demander
une hausse
aws freetier get-account-plan-state                            # crédits de départ

# Dépôt, code, modèle (§ 5 à 8)
git init · .gitignore · samconfig.toml · backend/ · frontend/ · template.yaml
sam validate --lint

# Mise en ligne (§ 9 et 10)
./scripts/bootstrap.sh
./scripts/deploy.sh

# Tests (§ 12 et 13)
(cd backend && npm test)
cd tests && npm ci && node convergence/convergence.mjs && node load/claim-race.mjs && node load/
credit-multidevice.mjs
./scripts/loadtest.sh up && ./scripts/loadtest.sh run 'bash load/peak.sh' && ./scripts/loadtest.sh
pull && ./scripts/loadtest.sh down

# Coûts (§ 14), puis fin (§ 15)
aws ce get-cost-and-usage --region us-east-1 ...
./scripts/destroy.sh
```

Annexe B · Où chaque règle du sujet est appliquée

Règle	Application concrète
Plan gratuit, une région	Compte Free plan (aucune facture possible) ; `region` dans `samconfig.toml` et dans les scripts ; services à l'usage, nuls au repos ; machine de test d'un type gratuit, supprimée après usage.
Aucune clé AWS	`aws login` (identifiants temporaires) ; rôles IAM pour les fonctions et la machine ; secrets applicatifs générés dans SSM Parameter Store ; vérification `grep AKIA` avant chaque envoi.
Moindre privilège	Un rôle par fonction, action par action et ressource par ressource (§ 8.8) ; buckets privés ; site lisible par la seule distribution CloudFront ; machine sans port ouvert ; preuve par `simulate-principal-policy`.
Fichiers dans S3	PDF dans `DataBucket/pdf/`, distribués par URL pré-signée de 60 s ; versions dans `versions/`.
Bonnes pratiques	Infrastructure as Code ; déploiement et suppression scriptés ; étiquette `project` ; journaux avec rétention ; chiffrement et HTTPS obligatoire ; tests unitaires avant chaque déploiement ; idempotence et réessais avec délai aléatoire ; Lambda arm64.

Sources

1. AWS CLI : installation et `aws login`, docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html et …/cli-configure-sign-in.html.

2. AWS SAM CLI : installation, docs.aws.amazon.com/serverless-application-model/latest/developerguide/install-sam-cli.html.
3. Régions d'un compte « projet », docs.aws.amazon.com/accounts/latest/reference/project-regions.html.
4. DynamoDB : expressions de condition et transactions, docs.aws.amazon.com/amazondynamodb/latest/developerguide/.
5. Lambda : concurrence réservée et filtrage d'événements, docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html et …/invocation-eventfiltering.html.
6. API Gateway : limitation des API HTTP ; CloudFront : Origin Access Control, docs.aws.amazon.com/apigateway/ et docs.aws.amazon.com/AmazonCloudFront/.
7. Dépôt du projet : README.md, docs/conception.md, docs/api.md, docs/rapport-tests.md, docs/couts.md, tests/results/.